Machine Coding: Live Clock

- **Problem:** Build a live digital clock that shows hours, minutes and seconds (12h with AM/PM or 24h) and the date, updates exactly when the second changes without drifting, can show any time zone (with a world-clock list driven by the same tick), and has a fullscreen mode via a button or the **F** key (ignored while typing). It must be cheap to run, correct across time zones and DST, and not leak timers or listeners.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Format, zones, fullscreen rules
Use cases + architecture + state	5 min	One `now` state, derived display
Ticking clock with `setInterval` (simplest)	6 min	Working clock
Drift-free tick aligned to the second + cleanup	10 min	Accurate timing
`Intl.DateTimeFormat` formatting, 12/24h, time zones	12 min	Correct everywhere
Fullscreen hook + F shortcut + world clocks	10 min	Full feature
Self-check + test + wrap-up	12 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"The only state is one `Date`: the current instant. Hours, minutes, AM/PM and every time zone are derived from it*

with `Intl.DateTimeFormat`*. I tick once per second, aligned to the start of the next second, so the display never lags or skips."*

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show `hh:mm:ss` + AM/PM (12h) or `HH:mm:ss` (24h), and the full date.
- Update every second, exactly when the second changes.
- Choose a time zone (local by default); show other zones as a world-clock list.
- Toggle 12h/24h.
- Fullscreen: a button and the **F** key; Esc (browser) exits; the button label follows the real state.

Non-functional requirements

Concern	What it means here
Performance	One re-render per second (not 100); one timer for all clocks; formatters cached.
Scalability	`Clock` is a pure display component; add zones without adding timers.
Error handling	Fullscreen can be refused (no gesture, iframe, unsupported): caught, no crash. Timers/listeners always cleaned up.
Accessibility	`<time dateTime>` semantics; **no** live region ticking every second; visible focus; `aria-pressed` on the 24h toggle; F ignored while typing or with modifiers.
Reusability	`useNow()`, `useFullscreen()`, pure `formatClock()`.
Extensibility	Analog face, alarms, stopwatch, locale selection.

Clarifying questions to ask the interviewer

1. 12h or 24h (or a toggle)? Show seconds? Show the date?
2. Whose time: the user's local time zone, a fixed one (e.g. Asia/Kolkata), or several?
3. Must it be accurate to the second (no drift), e.g. for an exam or auction page?
4. Is fullscreen required? Any keyboard shortcut?
5. Should it keep working in a background tab (timers get throttled)?
6. Any server-time requirement (the user's clock might be wrong)?

Assumptions we will build with

- Local zone by default; a selector + world list (Kolkata, London, New York, Tokyo).
- 12h default with a 24h toggle; seconds and date shown.
- Tick aligned to the next second boundary; re-sync when the tab becomes visible.
- Fullscreen via button + F (not in inputs, not with Ctrl/Cmd/Alt).
- Client clock is trusted (server-time sync is an extension).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Current local time immediately (no blank first second) + date
Each second	The display changes once, right as the second flips (no lag, no skipped seconds)
Click "24-hour"	`14:08:29`, no AM/PM; button shows pressed
Midnight in 24h	`00:00:00`, never `24:00:00`
Noon in 12h	`12:00:00 PM`
Select "New York"	Main clock shows New York time and date (can be a different day)
World clocks	All tick in sync with the main clock
Click Fullscreen / press F	Enters fullscreen; label "Exit fullscreen"; world list hidden
Press Esc in fullscreen	Browser exits; our label updates (`fullscreenchange`)
Press F while the select is focused	Nothing (not treated as a shortcut)
Ctrl+F	Browser find, not fullscreen
Tab hidden for minutes, then shown	Clock jumps to the correct time immediately
Fullscreen not allowed	Nothing breaks; button stays "Fullscreen"
DST change in a zone	`Intl` applies the right offset automatically

3. Design the architecture

Module	Responsibility
`hooks/useNow.js`	One `Date` state that updates at each second boundary; re-sync on visibility; cleanup
`hooks/useFullscreen.js`	`isFullscreen` from `fullscreenchange`; stable `toggle()` with error handling
`utils/formatClock.js`	Pure `formatClock(date, { timeZone, hour12 })` with cached `Intl.DateTimeFormat`s
`components/Clock.jsx`	Pure display: time, period, date for one zone
`App`	Owns `hour12` + selected zone; one `useNow()`; F shortcut; renders the main clock, controls, world list

```
App
├── now = useNow()                          ← single ticking source
├── { isFullscreen, toggle } = useFullscreen()
├── keydown "f" → toggle()  (not in inputs / with modifiers)
├── <Clock now timeZone={selected} hour12 size="large" />
├── controls: zone <select>, 24-hour toggle, Fullscreen button
```

```
└── world: ZONES.map → <Clock now timeZone hour12 size="small" />
Clock → formatClock(now, { timeZone, hour12 }) → Intl.DateTimeFormat (cached)
```

Start simple. First version: `useState(new Date())` + `setInterval(() => setNow(new Date()), 1000)` + `toLocaleTimeString()`. Then align the tick, switch to `formatToParts`, add zones, then fullscreen.

4. Design the state / data model

State	Where	Purpose
`now`	useNow	The current instant (`Date`)
`hour12`	App	12h vs 24h
`zone`	App	Selected IANA time zone (`"America/New_York"`)
`isFullscreen`	useFullscreen	Mirrors `document.fullscreenElement`

```
const now = useNow();                       // Date, updated each second
const [hour12, setHour12] = useState(true);
const [zone, setZone] = useState(LOCAL);    // Intl.DateTimeFormat().resolvedOptions().timeZone

// derived in Clock:
const { time, period, date } = formatClock(now, { timeZone, hour12 });
```

Senior note

- **One instant, many views:** the first version stored `hour`, `minutes`, `second` and `period` as four states: four setters per tick, and they could disagree mid-update. A single `Date` is the truth; everything else is formatting.
- **A `Date` is an instant, not a wall-clock time.** Time zones only matter when *displaying* it. Never "convert" a Date to another zone (that's what `new Date(now.toLocaleString(..., { timeZone }))` tries, and it creates a wrong instant).
- **Ownership:** the hook owns time; `App` owns user preferences; `Clock` owns nothing.

5. Design the data structures and contracts

Zones (IANA names, stable ids used as keys):

```
const ZONES = [
  { id: LOCAL, label: `Local (${LOCAL})` },
  { id: "Asia/Kolkata", label: "India (Kolkata)" },
  { id: "America/New_York", label: "New York" },
];
```

Formatting contract (pure):

```
formatClock(new Date("2026-11-02T00:05:09Z"), { timeZone: "UTC", hour12: true })
// → { time: "12:05:09", period: "AM", date: "Monday, November 2, 2026" }
formatClock(sameInstant, { timeZone: "America/New_York", hour12: true })
// → { time: "07:05:09", period: "PM", date: "Sunday, November 1, 2026" }
```

Hooks:

```
useNow() → Date
useFullscreen() → { isFullscreen: boolean, toggle: () => Promise<void>, supported: boolean }
```

DOM: `<time datetime="2026-11-02T00:05:09.000Z">12:05:09 AM</time>`, machine-readable instant + human text.

6. Trace the data flow

One tick at 21:37:59.990 local time:

```
useNow effect (mount): msToNextSecond = 1000 − (now % 1000) → setTimeout(tick, 10 ms)
  ↓ 21:38:00.002 tick → setNow(new Date()) → schedule next in 998 ms (re-aligned every time)
  ↓ App re-renders once
  ↓ each <Clock> → formatClock(now, zone) → cached Intl formatter → formatToParts
  ↓   { hour: "09", minute: "38", second: "00", dayPeriod: "PM" } → "09:38:00" + "PM"
  ↓ DOM: <time> text changes (tabular numbers, no jitter); world clocks change in the same frame
(tab hidden → timers throttled → on "visibilitychange" visible: clearTimeout + tick() immediately)
```

7. Important technical challenges

Challenge 1: accurate ticking (no drift, no wasted renders)

- *Problem:* The first version used `setInterval(getCurrentTime, 10)`: 100 updates and re-renders per second to show something that changes once per second. A plain `setInterval(…, 1000)` starts at a random offset (the display flips up to ~1 s after the real second) and drifts as timers fire late.
- *Solution:* Schedule each tick for the start of the next second, recomputed every time.

```
const msToNextSecond = () => 1000 - (Date.now() % 1000);
const tick = () => {
  setNow(new Date());
  timer = setTimeout(tick, msToNextSecond());
};
timer = setTimeout(tick, msToNextSecond());
return () => clearTimeout(timer);
```

Challenge 2: time zones done right

- *Problem:* `new Date(now.toLocaleString("en-US", { timeZone: "Asia/Kolkata" }))` turns a formatted string back into a Date in the *browser's* zone. It's a different instant, it breaks around DST changes, and parsing locale strings isn't guaranteed by the spec.
- *Solution:* Keep the real instant; format it per zone with `Intl.DateTimeFormat` + `formatToParts`, cached per (zone, 12/24h).

```
new Intl.DateTimeFormat("en-US", { timeZone, hour: "2-digit", minute: "2-digit", second: "2-digit",
                                   hourCycle: hour12 ? "h12" : "h23" })
const parts = Object.fromEntries(fmt.formatToParts(date).map((p) => [p.type, p.value]));
// time = `${parts.hour}:${parts.minute}:${parts.second}`, period = parts.dayPeriod
```

Challenge 3: event listeners and the F shortcut

- *Problem:* The first version's keydown effect had **no dependency array** (it re-ran after every render, 100×/s with the 10 ms timer), added the listener to `document` but removed it from `window` (so it was never removed: listeners piled up), and toggled on "f" even while typing in an input or pressing Ctrl+F.
- *Solution:* One effect with proper deps, the same target for add/remove, a stable `toggle` (`useCallback`), and guards.

```
useEffect(() => {
  const onKeyDown = (e) => {
    if (e.key.toLowerCase() !== "f" || e.repeat || e.ctrlKey || e.metaKey || e.altKey) return;
    if (isTypingTarget(e.target)) return;
    toggle();
  };
  window.addEventListener("keydown", onKeyDown);
  return () => window.removeEventListener("keydown", onKeyDown);
}, [toggle]);
```

Challenge 4: fullscreen state that matches reality

- *Problem:* `requestFullscreen()` returns a promise that **rejects** when not allowed (no user gesture, iframe without permission, unsupported): an unhandled rejection. Users can exit with Esc or browser UI, so a local `isFullscreen` flag toggled by our button gets out of sync.
- *Solution:* Read the truth from `document.fullscreenElement` on every `fullscreenchange` event, and wrap the calls in try/catch.

```
useEffect(() => {
  const onChange = () => setIsFullscreen(Boolean(document.fullscreenElement));
  document.addEventListener("fullscreenchange", onChange);
  return () => document.removeEventListener("fullscreenchange", onChange);
}, []);
const toggle = useCallback(async () => {
  try { document.fullscreenElement ? await document.exitFullscreen() : await
document.documentElement.requestFullscreen(); }
```

```
  catch { /* not allowed */ }
}, []);
```

8. Implement in the correct order

#	Step	Working result
1	`now` state + `setInterval(1000)` + `toLocaleTimeString()`	Ticking clock
2	Cleanup on unmount; initial value without waiting a second	No leaks, no blank
3	Align ticks to the second boundary (recursive `setTimeout`)	Accurate
4	`formatClock` with `Intl.DateTimeFormat` + `formatToParts`; tabular numbers	Clean formatting
5	12h/24h with `hourCycle` (h12/h23)	No "24:00" bug
6	`timeZone` option + zone selector	Any zone
7	`Clock` component; world list sharing one `now`	Many clocks, one timer
8	Cache formatters	Cheap per tick
9	`useFullscreen` (`fullscreenchange`, try/catch, stable toggle)	Reliable fullscreen
10	F shortcut with guards	Keyboard friendly
11	`visibilitychange` re-sync	Correct after background tabs
12	Node self-check (UTC, Kolkata, New York previous day, midnight/noon)	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
First render	Shows the real time immediately (`useState(() => new Date())`)
Each second	Exactly one update, right after the boundary
Midnight (24h)	`00:00:00` (`hourCycle: "h23"`)
Noon / midnight (12h)	`12:00:00 PM` / `12:00:00 AM`
Half-hour offsets (India +05:30)	Correct via `Intl`
Different date in another zone	The date line comes from the same zone
DST transitions	Handled by the IANA database in `Intl`
Background tab	Timers throttled; re-synced on `visibilitychange`
Computer sleep / system clock change	Next tick reads `Date.now()` again, so it's correct
Fullscreen refused	Caught; UI unchanged

State / edge case	Handling
Exit fullscreen with Esc	`fullscreenchange` updates the label
F in an input/select / with Ctrl/Cmd/Alt / key held	Ignored
Unmount	Timer, visibility, keydown and fullscreen listeners removed
Screen readers	Not spammed (no live region); `<time dateTime>` available
Local zone alias (e.g. `Asia/Calcutta`)	Valid IANA alias; shown as returned by the browser

10. Final structure

```
_16_live_clock/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css               ← global reset
    ├── App.jsx                 ← preferences, shortcut, layout
    ├── App.css                 ← clock styles, tabular numbers, fullscreen
    ├── utils/
    │   └── formatClock.js      ← cached Intl formatting (pure)
    ├── hooks/
    │   ├── useNow.js           ← aligned 1 s tick + visibility re-sync
    │   └── useFullscreen.js    ← fullscreen state + toggle
    └── components/
        └── Clock.jsx           ← display for one zone
```

11. Full working code

`src/utils/formatClock.js`

```
// Intl.DateTimeFormat is expensive to create, so cache one per (timeZone, hour12).
const cache = new Map();

function formatters(timeZone, hour12) {
  const key = `${timeZone}|${hour12}`;
  if (!cache.has(key)) {
    cache.set(key, {
      time: new Intl.DateTimeFormat("en-US", {
        timeZone,
        hour: "2-digit",
        minute: "2-digit",
        second: "2-digit",
        hourCycle: hour12 ? "h12" : "h23", // h23: midnight is 00, never "24"
      }),
```

```
      date: new Intl.DateTimeFormat("en-US", { timeZone, weekday: "long", year: "numeric", month:
"long", day: "numeric" }),
    });
  }
  return cache.get(key);
}

// One Date (an absolute instant) -> what a wall clock shows in `timeZone`.
export function formatClock(date, { timeZone, hour12 }) {
  const { time, date: dateFormat } = formatters(timeZone, hour12);
  const parts = Object.fromEntries(time.formatToParts(date).map((p) => [p.type, p.value]));
  return {
    time: `${parts.hour}:${parts.minute}:${parts.second}`,
    period: hour12 ? parts.dayPeriod : "",
    date: dateFormat.format(date),
  };
}

export const localTimeZone = () => Intl.DateTimeFormat().resolvedOptions().timeZone;
```

src/hooks/useNow.js

```
import { useEffect, useState } from "react";

// Current time as React state, updated exactly when the second changes.
// setInterval(…, 1000) drifts and can show a second late; we re-align on every tick instead.
export function useNow() {
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    let timer;
    const msToNextSecond = () => 1000 - (Date.now() % 1000);
    const tick = () => {
      setNow(new Date());
      timer = setTimeout(tick, msToNextSecond());
    };
    timer = setTimeout(tick, msToNextSecond());

    // Background tabs throttle timers; catch up immediately when the tab becomes visible again.
    const onVisibility = () => {
      if (document.visibilityState === "visible") {
        clearTimeout(timer);
        tick();
      }
    };
    document.addEventListener("visibilitychange", onVisibility);
```

```
    return () => {
      clearTimeout(timer);
      document.removeEventListener("visibilitychange", onVisibility);
    };
  }, []);

  return now;
}
```

src/hooks/useFullscreen.js

```
import { useCallback, useEffect, useState } from "react";

// Fullscreen state that follows the browser (Esc also exits fullscreen without our code).
export function useFullscreen() {
  const [isFullscreen, setIsFullscreen] = useState(() => Boolean(document.fullscreenElement));

  useEffect(() => {
    const onChange = () => setIsFullscreen(Boolean(document.fullscreenElement));
    document.addEventListener("fullscreenchange", onChange);
    return () => document.removeEventListener("fullscreenchange", onChange); // same target as add
  }, []);

  // Stable identity: the page re-renders every second, and listeners depending on `toggle` shouldn't
re-subscribe.
  const toggle = useCallback(async () => {
    try {
      if (document.fullscreenElement) await document.exitFullscreen();
      else await document.documentElement.requestFullscreen();
    } catch {
      // Not allowed (no user gesture, iframe without permission, unsupported): ignore.
    }
  }, []);

  return { isFullscreen, toggle, supported: Boolean(document.fullscreenEnabled) };
}
```

src/components/Clock.jsx

```
import { formatClock } from "../utils/formatClock";

// Pure display: the same `now` instant rendered in any time zone.
const Clock = ({ now, timeZone, hour12, label, size = "large" }) => {
  const { time, period, date } = formatClock(now, { timeZone, hour12 });

  return (
```

```
    <div className={`clock clock-${size}`}>
      {label && <p className="clock-label">{label}</p>}
      {/* Not a live region: announcing every second would be unbearable for screen-reader users. */}
      <time dateTime={now.toISOString()} className="clock-time">
        {time}
        {period && <span className="clock-period"> {period}</span>}
      </time>
      <p className="clock-date">{date}</p>
    </div>
  );
};

export default Clock;
```

src/App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";
import Clock from "./components/Clock";
import { useNow } from "./hooks/useNow";
import { useFullscreen } from "./hooks/useFullscreen";
import { localTimeZone } from "./utils/formatClock";

const LOCAL = localTimeZone();
const ZONES = [
  { id: LOCAL, label: `Local (${LOCAL})` },
  { id: "Asia/Kolkata", label: "India (Kolkata)" },
  { id: "Europe/London", label: "London" },
  { id: "America/New_York", label: "New York" },
  { id: "Asia/Tokyo", label: "Tokyo" },
];

const isTypingTarget = (el) => el.isContentEditable || ["INPUT", "TEXTAREA",
"SELECT"].includes(el.tagName);

function App() {
  const now = useNow(); // ONE ticking source shared by every clock on the page
  const [hour12, setHour12] = useState(true);
  const [zone, setZone] = useState(LOCAL);
  const { isFullscreen, toggle, supported } = useFullscreen();

  // "F" toggles fullscreen, but not while typing and not with Ctrl/Cmd/Alt shortcuts.
  useEffect(() => {
    const onKeyDown = (e) => {
      if (e.key.toLowerCase() !== "f" || e.repeat || e.ctrlKey || e.metaKey || e.altKey) return;
      if (isTypingTarget(e.target)) return;
      toggle();
```

```
    };
    window.addEventListener("keydown", onKeyDown);
    return () => window.removeEventListener("keydown", onKeyDown);
  }, [toggle]);

  const selected = ZONES.find((z) => z.id === zone) ?? ZONES[0];

  return (
    <main className={`page ${isFullscreen ? "fullscreen" : ""}`}>
      <h1 className="sr-only">Live clock</h1>

      <Clock now={now} timeZone={selected.id} hour12={hour12} label={selected.label} />

      <div className="controls">
        <label>
          Time zone{" "}
          <select value={zone} onChange={(e) => setZone(e.target.value)}>
            {ZONES.map((z) => (
              <option key={z.id} value={z.id}>
                {z.label}
              </option>
            ))}
          </select>
        </label>
        <button type="button" onClick={() => setHour12((h) => !h)} aria-pressed={!hour12}>
          24-hour
        </button>
        {supported && (
          <button type="button" onClick={toggle}>
            {isFullscreen ? "Exit fullscreen" : "Fullscreen"} <kbd>F</kbd>
          </button>
        )}
      </div>

      {!isFullscreen && (
        <section className="world" aria-label="World clocks">
          {ZONES.filter((z) => z.id !== selected.id).map((z) => (
            <Clock key={z.id} now={now} timeZone={z.id} hour12={hour12} label={z.label} size="small" />
          ))}
        </section>
      )}
    </main>
  );
}

export default App;
```

src/App.css

```
body { background: #0b1020; color: #e5e7eb; font-family: system-ui, sans-serif; }
.page { min-height: 100vh; display: flex; flex-direction: column; align-items: center; justify-content: center; gap: 28px; padding: 32px 16px; }

.clock { text-align: center; }
.clock-label { color: #9ca3af; font-size: 14px; text-transform: uppercase; letter-spacing: 0.1em; }
.clock-time { display: block; font-weight: 700; font-variant-numeric: tabular-nums; /* digits same width: no jitter */ }
.clock-large .clock-time { font-size: clamp(48px, 14vw, 140px); line-height: 1.1; }
.clock-period { font-size: 0.35em; color: #a5b4fc; }
.clock-date { color: #9ca3af; }
.clock-large .clock-date { font-size: clamp(16px, 2.5vw, 22px); }

.controls { display: flex; flex-wrap: wrap; gap: 10px; justify-content: center; align-items: center; }
select, button { padding: 8px 12px; border-radius: 8px; border: 1px solid #374151; background: #151b2e; color: inherit; font-size: 14px; cursor: pointer; }
button[aria-pressed="true"] { background: #4f46e5; border-color: #4f46e5; }
button:focus-visible, select:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
kbd { font-family: inherit; font-size: 12px; padding: 1px 5px; border: 1px solid #4b5563; border-radius: 4px; margin-left: 4px; }

.world { display: grid; grid-template-columns: repeat(auto-fit, minmax(170px, 1fr)); gap: 12px; width: min(800px, 100%); }
.clock-small { background: #151b2e; border: 1px solid #1f2940; border-radius: 12px; padding: 14px; }
.clock-small .clock-time { font-size: 26px; }
.clock-small .clock-date { font-size: 12px; }

.fullscreen .controls { opacity: 0.4; }
.fullscreen .controls:hover, .fullscreen .controls:focus-within { opacity: 1; }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: second-aligned ticking without drift, one timer for all clocks, correct time zones/DST/half-hour offsets, midnight/noon formatting, background-tab re-sync, fullscreen rejection and external exit, F shortcut guards, listener/timer cleanup, no screen-reader spam, no digit jitter.

Trade-offs to mention to the interviewer

- *Recursive* `setTimeout` *vs* `setInterval` *:* the timeout re-aligns every tick (no drift) at the cost of a few more lines; `setInterval` is simpler but drifts and starts at a random offset.
- `requestAnimationFrame` *vs timers:* rAF runs ~60×/s and pauses in background tabs; it's for smooth animations (an analog second hand), not for a once-per-second text update.
- *Client time vs server time:* the user's clock can be wrong. For auctions or exams, fetch the server time once, keep an offset, and display `Date.now() + offset` .
- `en-US` *formatting vs the user's locale:* fixed locale keeps the layout predictable; `undefined` locale respects user conventions (24h in many countries) but changes the layout.
- *Lifting* `now` *to App:* one timer and synchronized clocks, but the whole App re-renders every second; keep the clock subtree small, or isolate the ticking part in its own component.

What to add with more time

- Analog face (SVG hands rotated by `(seconds / 60) * 360` , with rAF for a smooth sweep).
- Server-time offset sync; alarms/timers; stopwatch.
- Locale selector and persisted preferences (12/24h, zone) in localStorage.
- Tests: `formatClock` (done as a self-check); `useNow` with fake timers (advance to the boundary → exactly one update); fullscreen hook with mocked `requestFullscreen` .

13. Counter questions the interviewer may ask

Requirements and design

Q1. Why not `setInterval(fn, 1000)` ?

It drifts (each callback can fire late, and delays add up) and starts at an arbitrary offset, so the display can lag the real second by up to ~1 s. A recursive `setTimeout` that waits `1000 - (Date.now() % 1000)` re-aligns on every tick.

Q2. Why was `setInterval(…, 10)` a problem?

It re-renders 100 times per second to show a value that changes once per second: wasted CPU and battery. Update only when the visible value can change.

Q3. Whose time should a clock show?

Usually the user's local time zone (`Intl.DateTimeFormat().resolvedOptions().timeZone`). Business pages (store hours, exams, events) should show the *business's* zone explicitly, with a label, because the user may be elsewhere.

State design

Q4. Why store one `Date` instead of hours/minutes/seconds?

They're all derived from the same instant. Separate states mean several updates per tick that can momentarily disagree, more renders, and more code. One source of truth, many formatted views.

Q5. Why lift `useNow()` to the App instead of each Clock having its own timer?

One timer keeps every clock perfectly in sync (they flip in the same frame) and costs one timer instead of N. The trade-off is that the parent re-renders each second, so keep it light.

Q6. Why does `useFullscreen` read `document.fullscreenElement` instead of keeping its own flag?

The user can leave fullscreen with Esc or the browser UI without going through our code. The document is the source of truth; the `fullscreenchange` event keeps React in sync.

Hard parts

Q7. What's wrong with `new Date(now.toLocaleString("en-US", { timeZone }))` ?

It parses a formatted string as if it were in the browser's local zone, creating a *different instant*. Arithmetic on it is wrong, it can break around DST transitions, and parsing locale strings is implementation-defined. Keep the real instant and format per zone.

Q8. Why `formatToParts` instead of `format` ?

`format` returns one string ("09:38:00 PM"), and you'd have to parse it to style the AM/PM separately. `formatToParts` returns typed pieces (`hour` , `minute` , `second` , `dayPeriod`) so you can lay them out and style them safely.

Q9. What's the `hourCycle` / "24:00" issue?

With `hour12: false` , some engines use the `h24` cycle for `en-US` and show midnight as `24:00:00` . `hourCycle: "h23"` guarantees 00–23; `"h12"` guarantees 1–12 with AM/PM.

Q10. What happens to timers in a background tab?

Browsers throttle them (often to once per second or less, sometimes much less after a few minutes). The clock might look stale when you come back, so we listen to `visibilitychange` and tick immediately when the tab becomes visible.

Q11. The add/remove listener targets didn't match. Why is that a bug?

`document.addEventListener` + `window.removeEventListener` removes nothing, because the listener lives on `document`. Combined with an effect that runs every render, it added a new keydown listener 100 times per second, so one F press toggled fullscreen many times. Always add and remove on the same target with the same function reference.

Implementation steps

Q12. How do you test a clock?

Make time controllable: fake timers + a fixed system time (`vi.useFakeTimers(); vi.setSystemTime(...)`), render, advance to the next second boundary, and assert exactly one update with the expected text. Test `formatClock` with fixed instants across zones (done here). Mock `requestFullscreen` / `fullscreenElement` for the hook.

Q13. How do you build an analog clock?

An SVG face with three `<line>` hands rotated around the center: seconds `s * 6°` , minutes `(m + s/60) * 6°` , hours `((h % 12) + m/60) * 30°` . For a smooth sweeping second hand use rAF with milliseconds; otherwise tick each second.

Q14. How do you avoid digits "jumping" sideways each second?

Most fonts have proportional digits ("1" narrower than "8"), so the text width changes. Use `font-variant-numeric: tabular-nums` (fixed-width digits) or a monospace font.

Performance and scale

Q15. The page re-renders every second. How do you keep that cheap?

Isolate the ticking state in a small component (the clock) so the rest of the page doesn't re-render; memoise heavy siblings; cache `Intl.DateTimeFormat` objects (they're expensive to create); avoid layout-heavy styles on the ticking text.

Q16. The user's system clock is 5 minutes wrong. How do you show the correct time?

Fetch the server time once (an API or the `Date` response header), compute `offset = serverTime − Date.now()` (adjusting for half the round-trip latency), and display `new Date(Date.now() + offset)` . Re-sync periodically.

Q17. Should screen readers announce the time every second?

No: a live region updating every second would make the page unusable. Leave it as normal content (readable on demand), or offer a "Read time" button / announce only on demand.

Trade-offs and "why not X?"

Q18. Why not `requestAnimationFrame` for the clock?

rAF fires ~60 times per second and pauses in background tabs. It's ideal for animating an analog second hand smoothly, but wasteful for a digital display that changes once per second.

Q19. Why not moment.js / date-fns-tz / Luxon?

`Intl.DateTimeFormat` already handles time zones, DST and locale formatting with zero bundle cost. Use a library for complex date math (adding months, durations, parsing many formats); Temporal (the new JS date API) will cover much of this natively.

Q20. Why does the fullscreen request sometimes fail?

Browsers require a user gesture (click/keypress) for `requestFullscreen`, may block it in iframes without `allow="fullscreen"`, and some devices (older iOS Safari for non-video elements) don't support it. That's why we check `document.fullscreenEnabled`, catch the rejection, and keep the UI in sync with `fullscreenchange`.

Q21. What is the Temporal API, and would it change this code?

Temporal is the modern replacement for `Date`: `Temporal.Now.zonedDateTimeISO("Asia/Kolkata")` gives a time-zone-aware value with clear types (Instant, ZonedDateTime, PlainTime). It would make zone conversion and arithmetic explicit and less error-prone; the formatting and ticking logic would stay the same.